

This document contains the implementation details and UI screenshots of the PYCS (Predict Your Crop Score) iOS application. It outlines the core features, user flows, and visualization components implemented using SwiftUI, along with their interaction with the backend machine learning services.

1.1 Core Features

1. Authentication

The PYCS application includes a secure user authentication system featuring intuitive Login and Signup pages (Figure 3.1 and 3.2). Currently, user email IDs and passwords are securely managed using the local file storage provided by Swift, ensuring a straightforward yet reliable authentication experience.

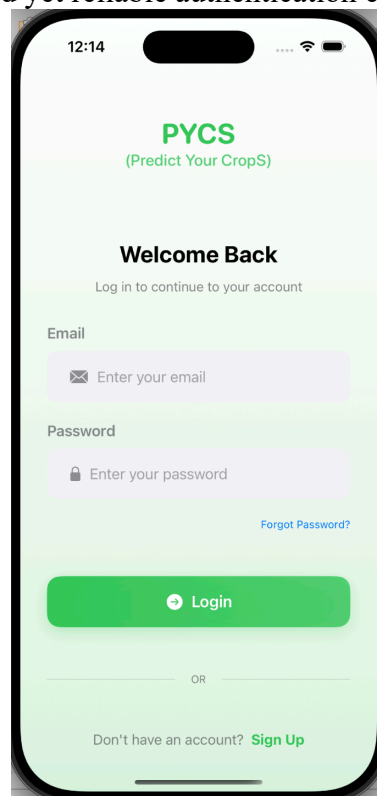


Figure 1.1 Login

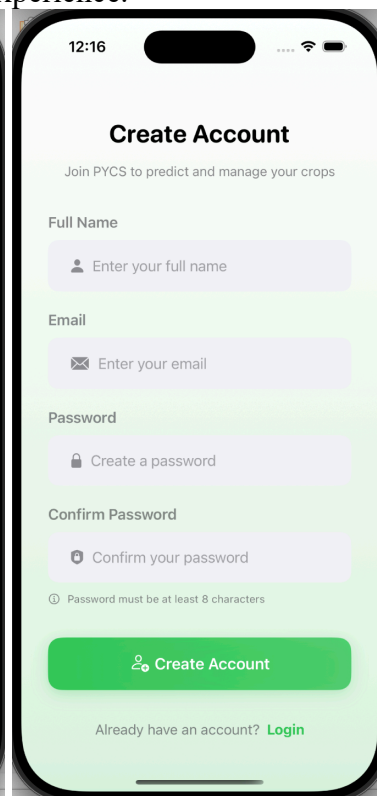


Figure 1.2 Signup

2. Dashboard Interface

Upon successful login, users encounter an intuitive Dashboard interface providing three main functionalities: (Figure 3.3)

- 1..1. Analyze Crop Data: Users can utilize the trained Machine Learning model to predict crop yields and generate new pickle files for future analysis and model updates. Additionally, this feature allows users to dynamically train new XGBoost models by uploading fresh datasets, which are then saved as unique pickle (.pkl) files for forecasting purposes.
- 1..2. Test Prediction: Provides functionality to perform forecasts using the newly generated pickle files to evaluate new models.

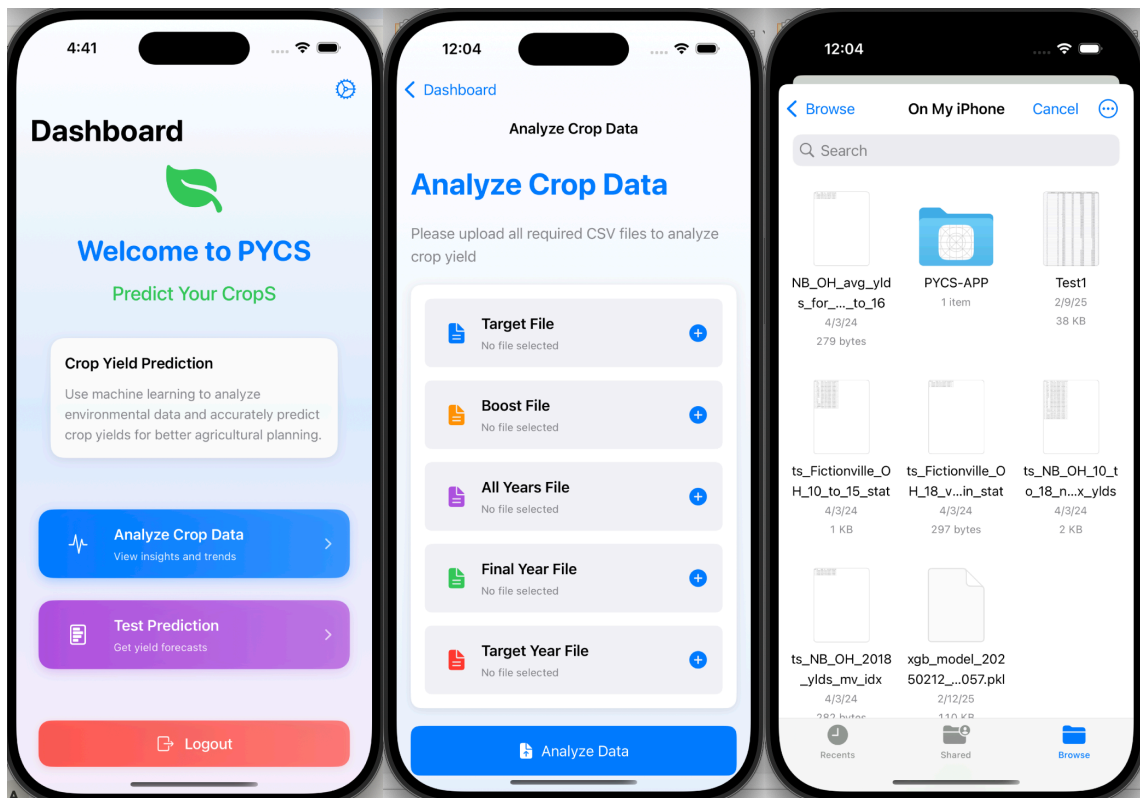


Figure 1.3 Dashboard

Figure 1.4 File Upload
Example

Figure 1.5 iOS File
Browser

3. **Data Management:** Data is managed through clear and accessible file management practices: (Figure 3.4 and 3.5)
 1. **CSV Files:** All data uploads strictly use CSV format to ensure compatibility and consistency. The input data specifically includes columns: index, yield, radiation, rain, avg_max_temp, avg_min_temp, and class.
 2. **PKL Files (Pickle files):** The trained model files are serialized and efficiently managed using Python's joblib library. Both CSV and pickle files are stored and accessed through the iOS File Manager, and these files are also accessible via iCloud Drive in the iOS Simulator. Swift's built-in DocumentPicker functionality facilitates smooth file uploads and selection within the app.
4. **Model Training:** A critical enhancement is the dynamic training capability, enabling users to train fresh XGBoost models directly through the PYCS interface. The process unfolds as follows:
 1. Every time a user initiates a forecast, a new XGBoost model is automatically trained using the latest uploaded dataset.
 2. The trained model is saved with a unique timestamped filename (e.g., xgb_model_tuned_20250414_153045.pkl) to maintain a clear version history.
 3. These models are instantly available for forecasting, performance comparison, and historical tracking.

5. Prediction Functionalities

1) **Analyze Crop Data:** The Analyze Data function trains and predicts a new XGBoost model via Flask API:

a) **Flask Endpoint (/upload):** Receives CSV files through POST requests.

b) **Input CSV Files:**

i) **target_file:** Features for prediction (radiation, rain, avg_max_temp, avg_min_temp)

ii) **boost_file:** Training data to fine-tune model

iii) **all_years_file:** Historical data

iv) **final_year_file:** Recent year data (used for prediction adjustment)

v) **target_year_file:** Actual yield values (for validation)

c) **Prediction Process:**

i) Trains the model using the input data and generates a new pickle file.

ii) Extracts and prepares input features.

iii) Executes actualTest function for:

(1) Yield predictions

(2) Adjustment with recent year's data (boosting accuracy)

(3) Performance metric calculation (MAE, RMSE, MAPE, SMAPE)

iv) Returns JSON responses with forecasts, metrics, and timestamps for visualization.

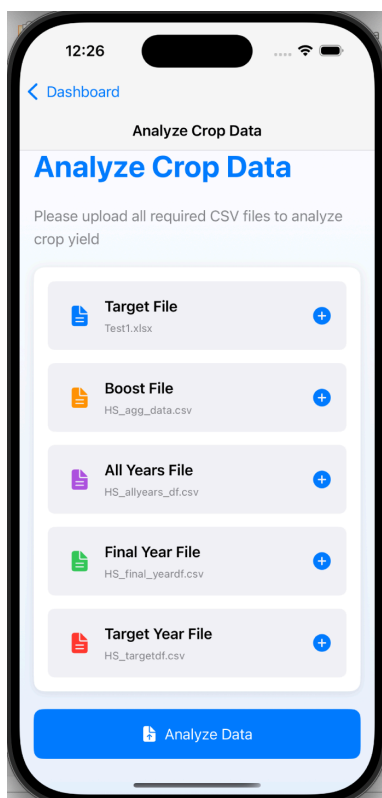


Figure 1.6: Uploading Required Input Files

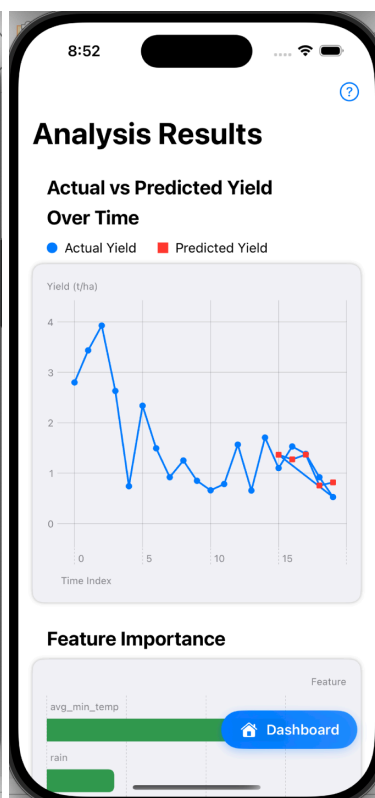


Figure 1.7: Yield Prediction

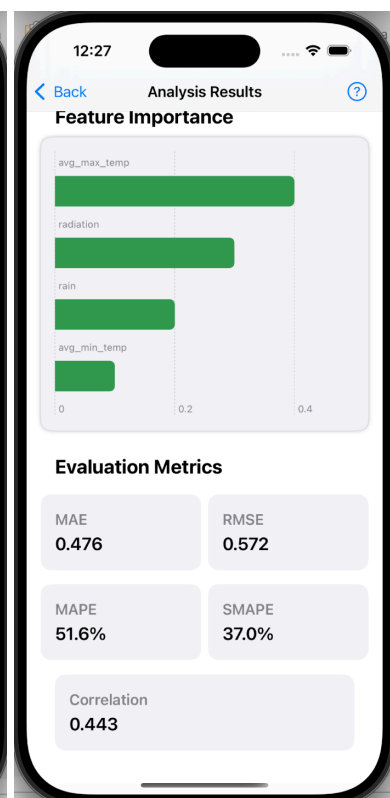


Figure 1.8: Feature Importance and Evaluation Metrics

- 2) Generation of new pickle files:- This is a part of the ‘Analyze Crop Data’ functionality. It processes the model's predictions with the provided data to generate new pickle files.
- a) Flask Endpoint (/upload): Receives new training CSV data from users.
 - b) Feature Processing Pipeline:
 - i) Trains a fresh XGBoost model based on the provided data.
 - ii) Generates unique serialized model files (.pkl) saved via joblib.dump().
 - iii) Stores preprocessing components (scalers, selected features) with models for a complete prediction pipeline.

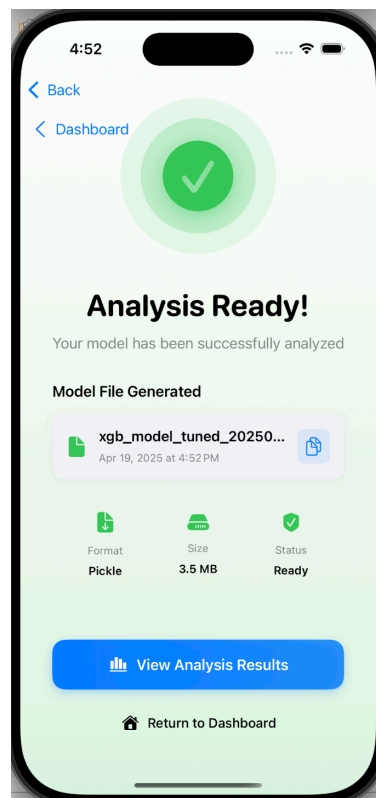


Figure 3.9: New Pickle file generation screen

- 3) Test Prediction
- This functionality allows users to generate forecasts by the evaluation of newly created models:
- a) Flask Endpoint (/predict): Accepts CSV files containing new weather data.
 - b) Prediction Steps:
 - i) Loads specified/newly generated serialized model.
 - ii) Processes input data, matching column format.
 - iii) Performs predictions on provided data.
 - iv) If actual yield values are present, calculates performance metrics (MAE, RMSE, SMAPE).
 - v) Provides results and metrics in consistent JSON format for easy consumption by the app UI.

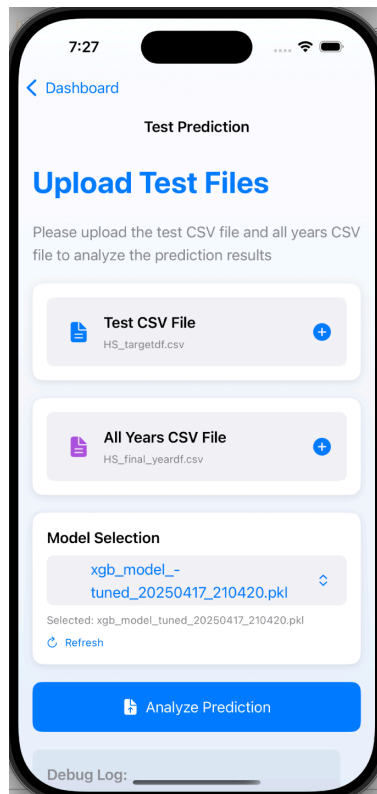


Figure 1.10: Upload Screen for predictions

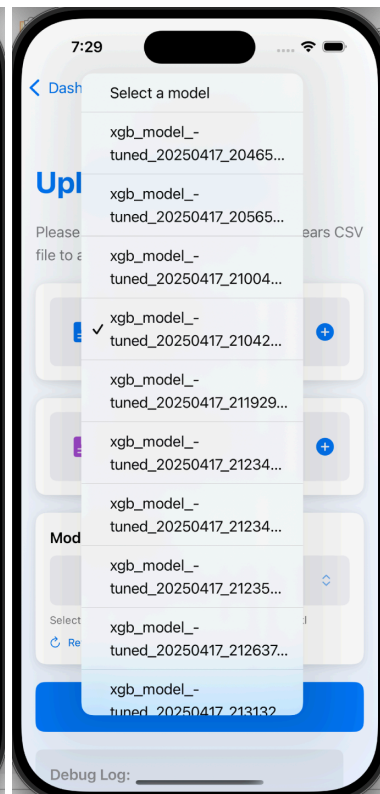


Figure 1.11: List to choose from existing pickle files

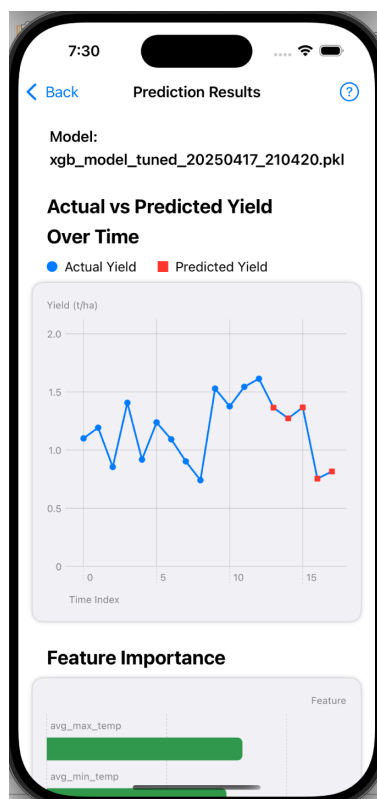


Figure 3.12: Yield chart

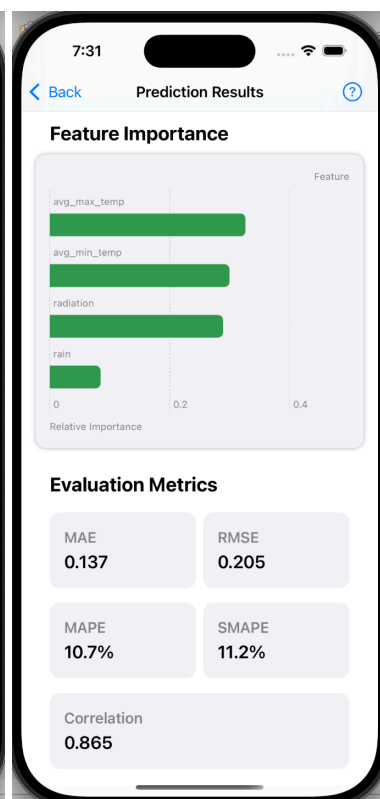


Figure 3.13: Feature Importance chart

6. Results Visualization: The app clearly and effectively visualizes prediction outcomes using various graphical formats:
 - 1) Yield Prediction Comparison: Presents actual versus predicted yields using detailed, visually intuitive comparison charts to highlight prediction accuracy clearly.
 - 2) Feature Importance: Displays clear graphical charts indicating environmental factors (e.g., radiation, rainfall, temperature) ranked by their significance in yield predictions, helping farmers prioritize management decisions.
 - 3) Evaluation Metrics: Comprehensive metrics visualized include:
 - a) Mean Absolute Error (MAE)
 - b) Root Mean Squared Error (RMSE)
 - c) Mean Absolute Percentage Error (MAPE)
 - d) Symmetric Mean Absolute Percentage Error (SMAPE)
 - e) Correlation Score

3.2 UI/UX Considerations

The app design emphasizes usability and ease-of-understanding with specific improvements based on user feedback:

1. Colorful, Clear Visualizations: Charts and visual elements utilize vibrant, distinguishable colors for clarity and ease of interpretation. Interactive visualizations allow horizontal scrolling, making detailed data inspection effortless and intuitive.
2. Integrated Help Button (Newly Added): A "Help" button provides brief explanations of evaluation metrics (MAE, RMSE, MAPE, SMAPE, Correlation), enhancing understanding and supporting effective data-driven decisions for users with limited technical expertise.

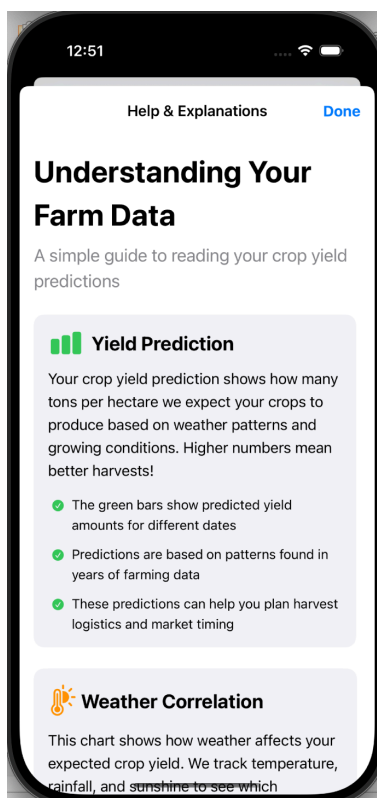


Figure 1.14: Help View

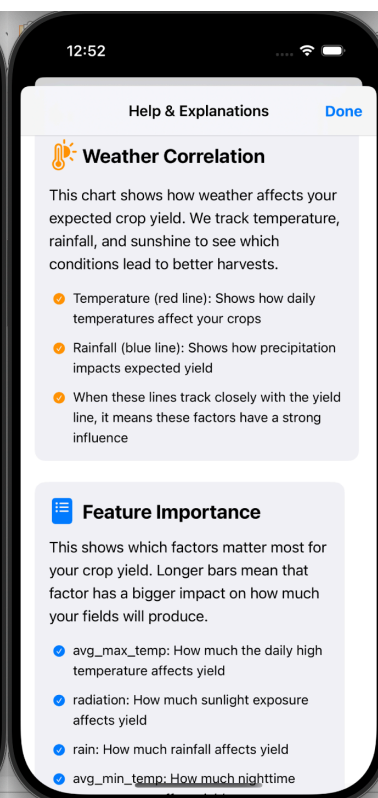


Figure 1.15: Help View

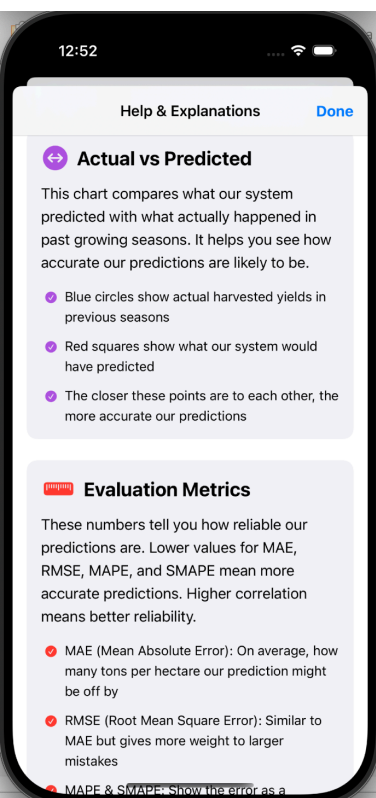


Figure 1.16: Help View